

Be part of a better internet. [Get 20% off membership for a limited time](#)

Memory requirements for fine-tuning Llama 2



Sri Ranganathan Palaniappan · [Follow](#)

Published in Polo Club of Data Science | Georgia Tech

6 min read · Apr 16, 2024



Listen



Share

... More

TL;DR: *Fine-tuning large language models like Llama-2 on consumer GPUs could be hard due to their massive memory requirements. However, techniques like Parameter Efficient Fine-Tuning (PEFT) methods, specifically QLoRA, can significantly **reduce the memory footprint by up to 90% to around 9–14GB** by quantizing the model to **4-bit precision** and training only a small fraction (0.1–1%) of the total parameters. **This makes fine-tuning more accessible and affordable, even on the free version of Google Colab with 16GB GPU memory!***

Table of Contents

- [Why Llama-2 \(and 7B-chat\)?](#)
- [Naively fine-tuning Llama-2 7B takes 110GB of RAM!](#)
- [PEFT lowers memory requirements by 90%](#)
 1. [Low Rank Adaptation \(LoRA\) for efficient fine-tuning](#)
 2. [Making fine-tuning more efficient: QLoRA](#)
- [How does QLoRA reduce memory to 14GB?](#)

Why Llama-2 (and 7B-chat)?

Llama-2, released by Meta in 2023, is one of the most widely used **open-source** Large Language Models (LLMs) today. These models belong to a class called Foundation Models, models which are trained on massive amounts of data (~ 2 trillion tokens) and can be fine-tuned for specific tasks. The Llama-chat models are one such example of foundation models finetuned for dialogue use cases, like creating chatbots using LLMs. The **smallest** Llama 2 chat model is **Llama-2 7B Chat**, with 7 billion parameters. It's a powerful and accessible LLM for fine-tuning because with fewer parameters it is an ideal candidate for starting out with fine-tuning.

Naively fine-tuning Llama-2 7B takes 110GB of RAM!

Even fine-tuning small models like Llama-2 7B on regular consumer GPUs can be challenging due to the significant memory requirements because of the following reasons:

1. **Memory Requirements:** Llama-2 7B has 7 billion parameters and if it's loaded in full-precision (float32 format-> 4 bytes/parameter), then the total memory requirements for loading the model would be $\text{numberOfParams} * \text{bytesPerParam} = 7 \text{ billion} * 4 = 28\text{GB of memory}$. Given that many consumer GPUs/ free versions of software like Google Colab or Kaggle [4] have memory constraints (e.g., NVIDIA T4 16GB on Google Colab), the model cannot even be loaded!
2. **Fine-Tuning memory requirements:** In the case of full fine-tuning with the regular 8bit Adam optimizer using a half-precision model (2 bytes/param), we need to allocate per parameter: 2 bytes for the weight, 2 bytes for the gradient, and 12 bytes for the Adam optimizer states [4]. This results in a total of 16 bytes per trainable parameter,

requiring over 110GB of GPU memory!! This would require at least 3A40s with 48GB GPU VRAM, which would mean fine-tuning wouldn't be accessible by public.

PEFT lowers memory requirements by 90%

Parameter Efficient Fine Tuning methods (PEFT) are used to drastically reduce the number of trainable parameters of the LLM model while maintaining performance [6].

A very popular PEFT method used for fine tuning LLMs is **Low Rank Adaptation (LoRA)** which drastically reduces the number of parameters to be modified [5].

Low Rank Adaptation (LoRA) for efficient fine-tuning

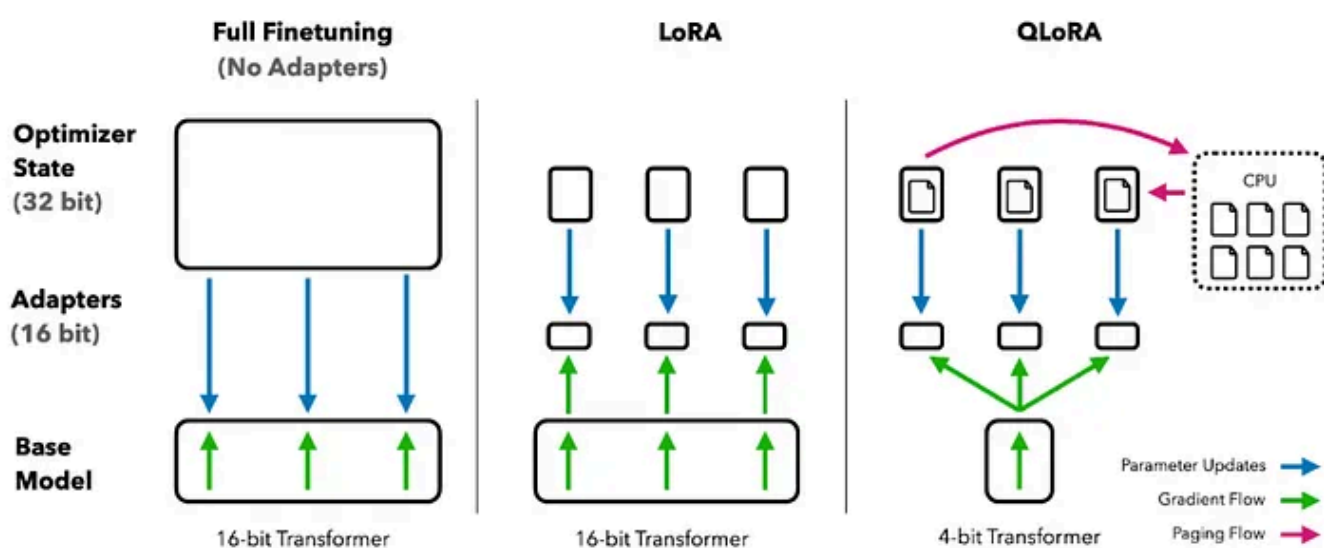
To make fine-tuning more efficient, LoRA decomposes the large weight matrix into two smaller, low-rank matrices which can be trained to adapt to the new data while keeping the overall number of changes low.

This approach of training only a small percentage of the total parameters' weights using LoRA can result in fine-tuned models which are comparable to the performance of fully fine-tuned models while requiring a fraction of the compute resources.

Making fine-tuning more efficient: QLoRA

QLoRA is a fine-tuning technique that combines a high-precision computing technique with a low-precision storage method [1]. This helps keep the loaded model size small while still making sure the model is still highly performant and accurate.

QLoRA involves 4-bit quantization (**4-bit NormalFloat (NF4) quantization**) along with LoRA- which helps with fine-tuning using lower memory requirements and in correcting minimal, residual quantization errors.



Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers like AdamW to handle memory spikes [1]. Image credit: [1]

How does QLoRA reduce memory to 14GB?

Below is the calculation to determine the memory requirements for fine tuning Llama-2-7B with QLoRA.

Memory requirement for loading the model: The Llama-2 7B base model has about 7 billion parameters (although precisely 6.7B), and each parameter is quantized to 4 bits (0.5 bytes). Hence, loading the model would take about **3.5 GB** ($\approx 7 \text{ billion parameters} \times 0.5 \text{ bytes}$).

Memory requirement per trainable parameter consists of:

- Weight: 0.5 bytes
- LoRA parameters: 2 bytes
- AdamW optimizer states: 2 bytes
- Gradients (always in fp32): 4 bytes
- Activation: variable (depends on factors like sequence length, hidden size and batch size)

Therefore, the memory per trainable parameter is **8.5 bytes** ($\approx 0.5 + 2 + 2 + 4$)

Total memory requirement for trainable parameters: Since LoRA results in an average of 0.4-0.7% [7] trainable parameters, assuming that there are 0.6% of trainable parameters, the total memory requirement for trainable parameters is:

Trainable parameters memory

= Memory per parameter * parameters

= 8.5 bytes * 42 million (0.6% of 7B parameters)

≈ 0.36 GB

Total memory requirement for QLoRA training: The total memory requirement for QLoRA training is around 4 GB, which includes the memory for the base model (**≈ 3.5 GB**) and the memory for trainable parameters **≈ 0.36 GB**, resulting in a total training memory requirement of about **$\approx 4-5$ GB** (depending on the number of trainable parameters).

Memory required for inference: If we load the base model in 16-bit precision and merge the LoRA weights of the fine-tuned model, we would **at-most use 14 GB** of GPU memory for a sequence length of 2048. This memory cost is derived from loading the model in float16 precision and includes activations, temporary variables and hidden states[3], which are always in full-precision (float32) format and depend on many factors including sequence length, hidden size and batch size[2].

Total memory requirements: So, the total memory requirement for QLoRA training with a 4-bit base model and mixed-precision mode, including loading the 32-bit model for inference, would be almost **≈ 14 GB** depending on the sequence length.

Thus, we can see that using quantization techniques like QLoRA along with PEFT can **significantly reduce memory requirements by up to 90%**, thereby making fine tuning more accessible and affordable!

Credits

- [Sri Ranganathan Palaniappan](#), CS undergrad student at Georgia Tech.
- [Mansi Phute](#), CS masters student at Georgia Tech.
- [Seongmin Lee](#), PhD student at Georgia Tech.
- [Polo Chau](#), Associate Professor at Georgia Tech.

Sources:

[1] Tim Dettmers. Artidoro Paononi. Ari Holtzman. and Luke Zettlemoyer. "OLoRA: Efficient Finetuning of Quantized

Open in app ↗

Medium

Search



https://muggingface.co/docs/transformers/peft_train_gpu_one#anatomy-of-models-memory.

[1] Hugging Face. "Anatomy of Models Memory." Accessed March 31, 2024.

https://huggingface.co/docs/transformers/perf_train_gpu_one#anatomy-of-models-memory.

[2] Hugging Face. "A standard AdamW uses 8 parameters for each weight tensor." Accessed March 31, 2024.

https://huggingface.co/docs/transformers/v4.23.1/en/perf_train_gpu_one#:~:text=A%20standard%20AdamW%20uses%208,all%20optimizer%20states%20are%20quantized.

[3] Dell Technologies. "LLAMA-2: Efficient Fine-tuning Using Low-Rank Adaptation (LoRA) on Single GPU." Accessed March 16, 2024. <https://infohub.delltechnologies.com/en-US/p/llama-2-efficient-fine-tuning-using-low-rank-adaptation-lora-on-single-gpu/>.

[4] PyTorch Team. "Finetune Large Language Models in PyTorch." PyTorch Blog, March 16, 2024.

<https://pytorch.org/blog/finetune-llms/>.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models." arXiv preprint arXiv:2106.09685, 2021.

[6] Hugging Face. "Trl-PEFT: An Efficient Approach for Fine-tuning Large Language Models." Accessed March 22, 2024. <https://huggingface.co/blog/trl-peft>.

[7] Databricks Team. "Efficient Fine-Tuning of Large Language Models: A Guide to LoRA." Databricks Blog, 2024.

<https://www.databricks.com/blog/efficient-fine-tuning-lora-guide-llms>.

LLM

Deep Learning

AI

Machine Learning

Generative AI Tools



Follow

Written by Sri Ranganathan Palaniappan

6 Followers · Editor for Polo Club of Data Science | Georgia Tech

CS @ Georgia Tech

More from Sri Ranganathan Palaniappan and Polo Club of Data Science | Georgia Tech



 Sri Ranganathan Palaniappan in Polo Club of Data Science | Georgia Tech

Fine-tune LLMs for free on custom text data: A Step-by-step Tutorial

Table of Contents

Jul 15  120



 Anthony Peng in Polo Club of Data Science | Georgia Tech

Multi-GPU Training in PyTorch with Code (Part 3): Distributed Data Parallel

We discussed single-GPU training in Part 1 and multi-GPU training with DP in Part 2. In Part 2, we found DP is incompatible with GPUs w/o...

Jul 7, 2023  70





Anthony Peng in Polo Club of Data Science | Georgia Tech

Multi-GPU Training in PyTorch with Code (Part 4): Torchrun

We discussed single-GPU training in Part 1, multi-GPU training with DP in Part 2, and multi-GPU training with DDP in Part 3. We will...

Jul 7, 2023 🖱 73



Anthony Peng in Polo Club of Data Science | Georgia Tech

Multi-GPU Training in PyTorch with Code (Part 2): Data Parallel

In Part 1, we successfully trained a ResNet34 on CIFAR10 using a single GPU. In this article, we will explore how to launch the training on...

Jul 7, 2023 🖱 112 💬 1



[See all from Sri Ranganathan Palaniappan](#)[See all from Polo Club of Data Science | Georgia Tech](#)

Recommended from Medium



 Dr. Walid Soula in Kinomoto.Mag AI

Supervised Fine-Tuning Tips for your LLM projects

Improve your language model's performance with these expert SFT tips from Philipp Schmidt.

★ Jun 2 🖱 1





Jason TC Chuang in aidatatools

Fine-Tuning LLM: Apple Studio M2 Ultra 192GB vs.

Fine-tuning LLM with NVIDIA GPU or Apple NPU (collaboration between the author, Jason and GPT-4o)



May 30



2



1

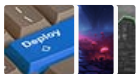


Lists



Natural Language Processing

1592 stories · 1151 saves



Predictive Modeling w/ Python

20 stories · 1400 saves



Practical Guides to Machine Learning

10 stories · 1693 saves



The New Chatbots: ChatGPT, Bard, and Beyond

12 stories · 426 saves



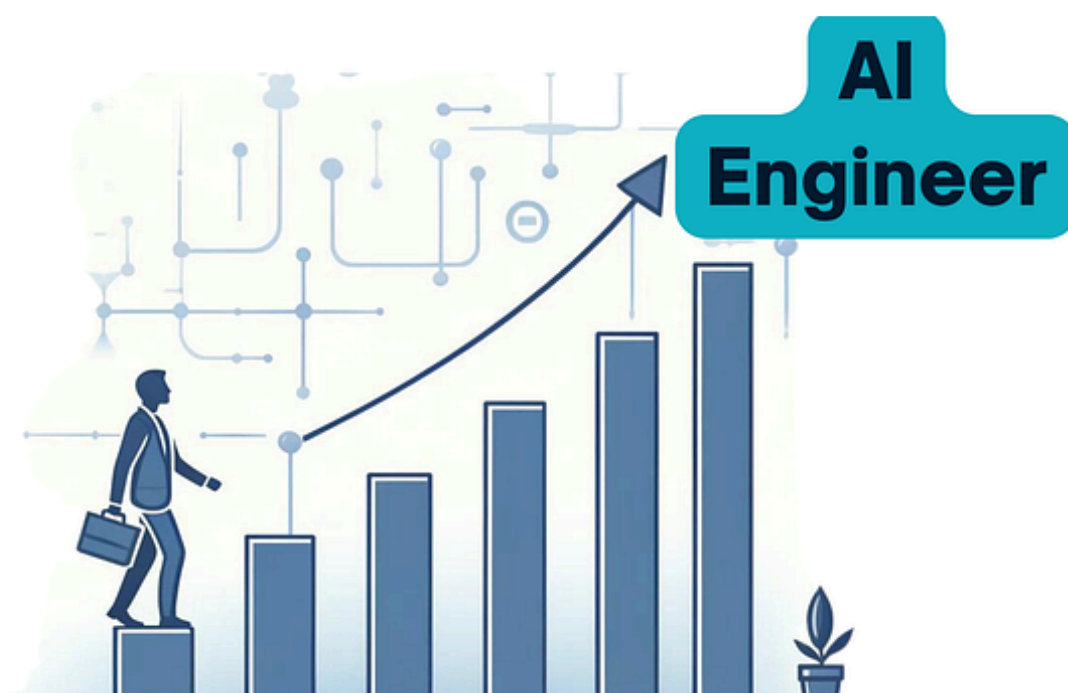
 Nedved Yang in The Constellar Digital&Technology Blog

Geek Out Time: Playing with a Machine Learning Pipeline with PyTorch and MLflow

I used Airflow many years ago in my first machine-learning project when I was at GFG. This week, I decided to experiment with a few other...

★ May 22

🔖 ...



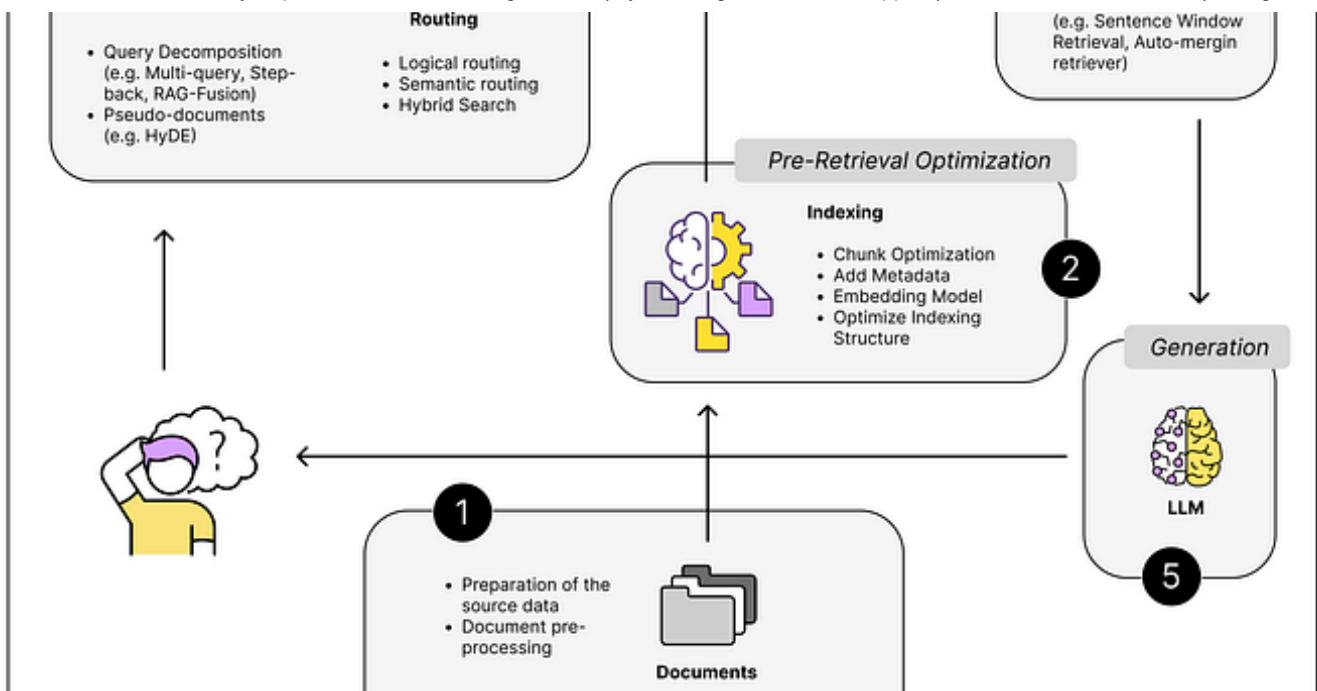
 Kris Ograbek in AI Advances

If I started learning AI Engineering in 2024, here's what I would do.

The exact path I would choose.

★ Jun 3 👍 843 💬 15

🔖 ...

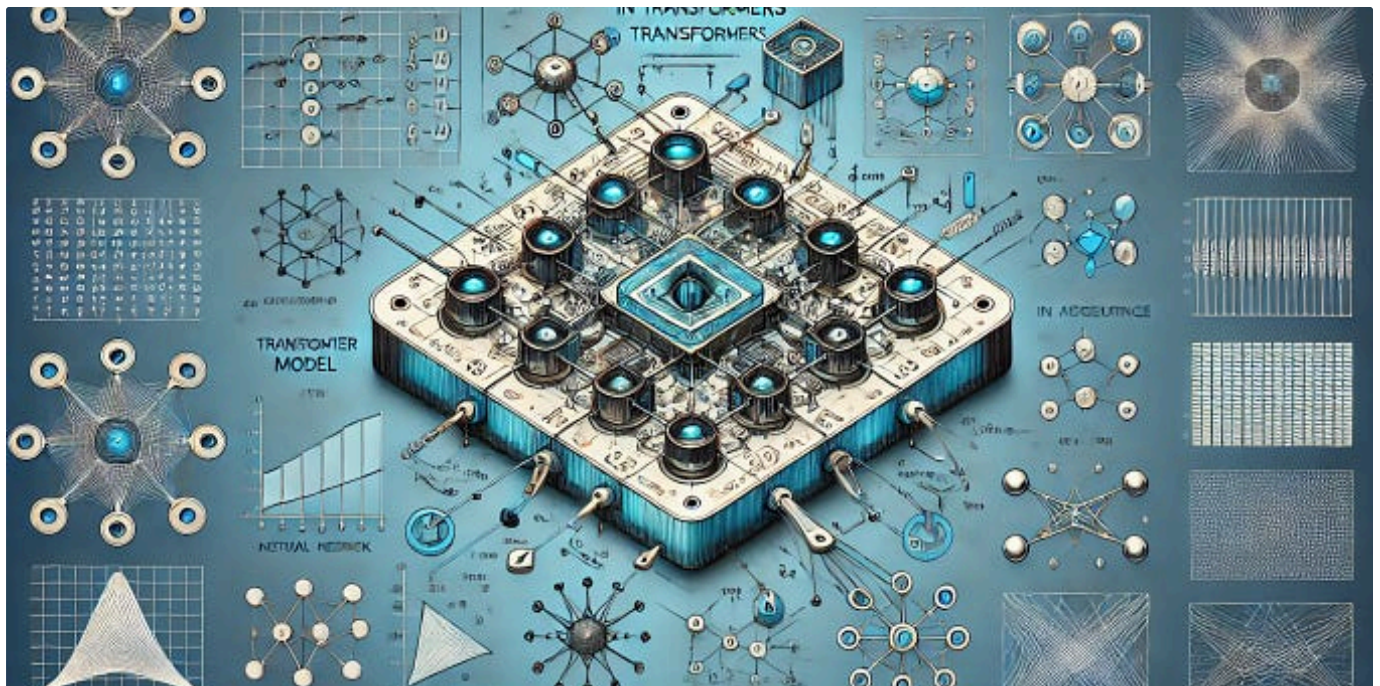


 Dominik Polzer in Towards Data Science

17 (Advanced) RAG Techniques to Turn Your LLM App Prototype into a Production-Ready Solution

A collection of RAG techniques to help you develop your RAG app into something robust that will last

★ Jun 26 🙋 1.99K 💬 20



 Cristian Leo in Towards Data Science

The Math Behind Multi-Head Attention in Transformers

Deep Dive into Multi-Head Attention, the secret element in Transformers and LLMs. Let's explore its math, and build it from scratch.

★ Jul 16 🤝 289 💬 3

See more recommendations